

Tile-Level Redundancy in LLMs — Findings (Qwen3-4B)

One line: You can delete about **5% of the model** (as 32×32 weight-tiles) before its real-task ability starts to break — *far* less than perplexity implies — and the thing that makes pruning work is **repairing the surviving weights, not cleverly choosing which tiles to cut**.

Model: Qwen/Qwen3-4B. Tile size: 32×32 (project standard, Rathore §2.4). Dense WikiText perplexity = 13.22. Downstream ability measured on HellaSwag / PIQA / ARC-Easy via lm-eval, normalised to above-chance range: $retained = (acc - chance) / (acc_{dense} - chance)$.

How to read this doc

Every finding below is written as **What we test** → **Why we ran it** → **Result** → **Reliability**. The *Reliability* tag is the single most important label:

- **downstream-anchored** — judged on real task accuracy. Trust these most.
- **control-relative** — a comparison against a matched control (e.g. random tiles) that shares the same biases, so the *ranking* is trustworthy even if absolute numbers aren't.
- **perplexity-absolute** — rests on raw perplexity; directionally useful but carries the caveat that perplexity misleads (that's Finding 2).

Data note: all headline numbers are at the corrected **32×32** tile size. A prior bug ran some controls at 64×64 ; those were re-run (commit [a1be30e](#)). A few findings still rest on archived 64×64 "shape evidence" — flagged explicitly where they occur.

The four things worth saying in the presentation

1. **~5% is the real redundancy budget** (downstream-anchored) — perplexity suggested 40–70%, and it was wrong.
 2. **Perplexity is not just a loose proxy — it is *gameable***. You can push perplexity *below dense* while the model gets measurably worse on real tasks.
 3. **Repair is the whole method**. Which tiles you delete barely matters; reconstructing the surviving weights is what preserves ability. Calibrated tile-selection loses to a coin flip at the usable operating point.
 4. **Redundancy is not a per-part property**. Pieces that are safe alone are not safe together, and safety is the *interaction* of depth $\times$ matrix-type, not either alone.
-

Reliability at a glance

Finding	Rests on	Status
1 — ~5% redundancy ladder	downstream-anchored	✅ holds
2 — perplexity is nonlinear	downstream-anchored	✅ holds
2b — perplexity is <i>gameable</i>	downstream drop tile-32; ppl-gaming tile-64	⚠️ holds, but "one model" pairing is cross-tile (see caveats)
3 — Policy B games its own metric	downstream-anchored	✅ holds
4 / 4e — repair >> selection	control-relative (same-tile ablation)	✅ holds-stronger (clean tile-32 paired ablation)
4b — repair backfires at final MLP	control-relative	✅ holds
4c / 4d — selection loses to random	control-relative (corrected tile-32 floor)	✅ holds-stronger
5 — isolation does not compose	downstream-anchored	✅ holds
6 / 6b — depth concentration	control-relative; now tile-32 ; exploratory	⚠️ monotonic (no interior optimum); confounds disclosed
7 — depth × matrix interaction	control-relative; now tile-32 , perplexity-only, marginal	⚠️ holds as a marginal perplexity-sensitivity map
8 — magnitude < random	control-relative (vs random control)	✅ holds
9 — damage sub-additive in-layer, compounds across	perplexity-absolute	⚠️ direction holds, carries caveat

Findings

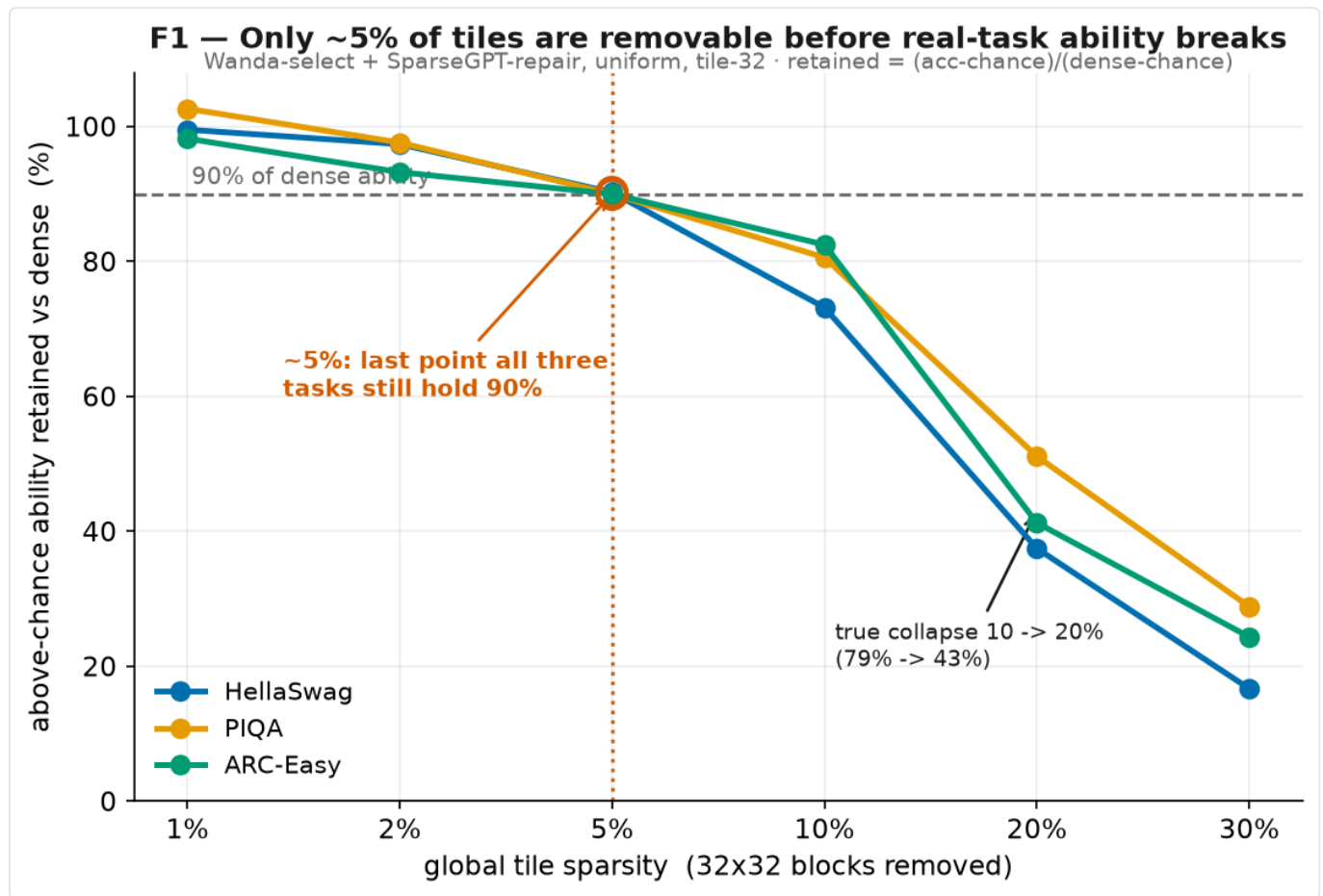
1 — About 5% of tiles are removable before real ability breaks (downstream-anchored)

What we test. How much of the model we can delete in 32×32 tiles before it fails on actual reasoning tasks — not just on a text-prediction score. We prune 1/2/5/10/20/30% with `sparsegpt_recon` (SparseGPT-select + SparseGPT-repair) and measure HellaSwag / PIQA / ARC-Easy. (Note: Finding 4 shows `sparsegpt_recon` is actually the weakest of the three repair variants at 5% — all repair variants cluster near dense, so the ~5% number is unaffected, but this ladder is not "the best method.")

Why we ran it. Our perplexity curves implied 40–70% was removable. Perplexity is only a proxy; we wanted the honest number judged on abilities a person cares about.

Result. ~5% of tiles can be removed while keeping ~90% of above-chance ability — and it lands at ~90% on all three tasks (the exact 90/90/90 is a rounding coincidence, not a law). **5% is the capability-preserving operating point** (~90% retained); it is not a cliff — 10% is still *usable-but-degraded* (~79%), and the real collapse is between 10% and 20% (79% → 43%). This 5%-vs-10% distinction matters for Findings 4c/4d/8, whose "selection loses to a coin flip" claim is scoped specifically to the 5% capability-preserving point.

Numbers (retained above-chance ability, HellaSwag/PIQA/ARC-Easy; tile-32): 1% → 100/103/98; 2% → 97/98/93; **5% → 90/90/90 (avg 90%)**; 10% → 73/81/82 (avg 79%); 20% → 37/51/41 (avg 43%); 30% → 17/29/24 (avg 23%). Paired WikiText ppl: 5%=15.62, 10%=18.78, 20%=30.95, 30%=61.56 (all tile-32).



retained ability vs sparsity, three tasks, 90% line + 5% marker.

2 — Perplexity is a *nonlinear* proxy, dangerous exactly where you'd rely on it ★ (downstream-anchored)

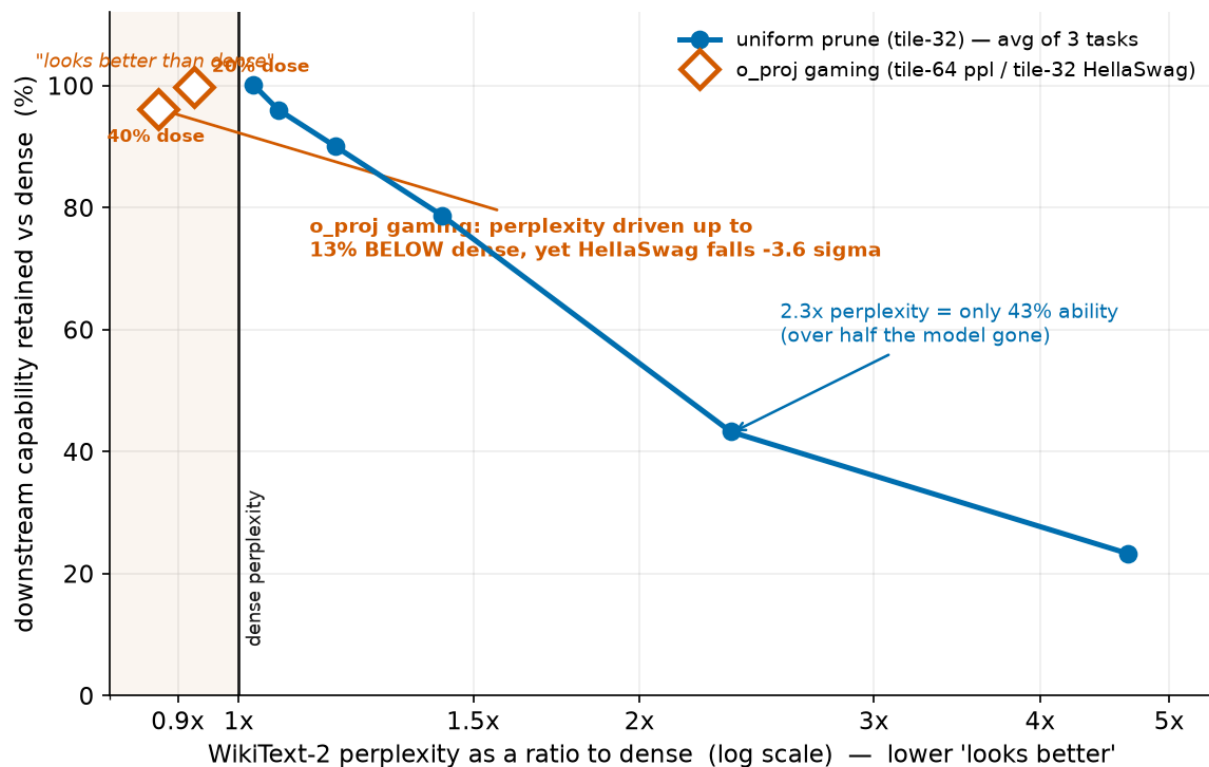
What we test. Whether WikiText perplexity actually tracks real ability across the sparsity range.

Why we ran it. Perplexity is the fast, cheap number we'd instinctively use to rank methods. If it flatters an aggressive setting, any perplexity-only headline is unsafe.

Result. Perplexity never points the *wrong* way, but the mapping to capability is brutally nonlinear. At 5% ppl reads 1.18× dense and the model keeps ~90% of ability; at 20% ppl reads only 2.34× ("about twice as bad") yet **over half the model's ability is gone** (43% retained); at 30%, 4.66× leaves 23%. So perplexity is fine in the usable regime (≤5%) and dangerously flattering exactly where you'd use it to justify aggressive pruning.

Numbers (tile-32): ppl-ratio → retained-avg: 1.18×→90%, 1.42×→79%, 2.34×→43%, 4.66×→23%.

F2/F2b — Perplexity is nonlinear, and o_proj can push it below dense while the model gets worse



(shared with 2b).

2b — Perplexity is *adversarially gameable*: push it below dense while the model gets worse ★★ (mixed tile — see caveat)

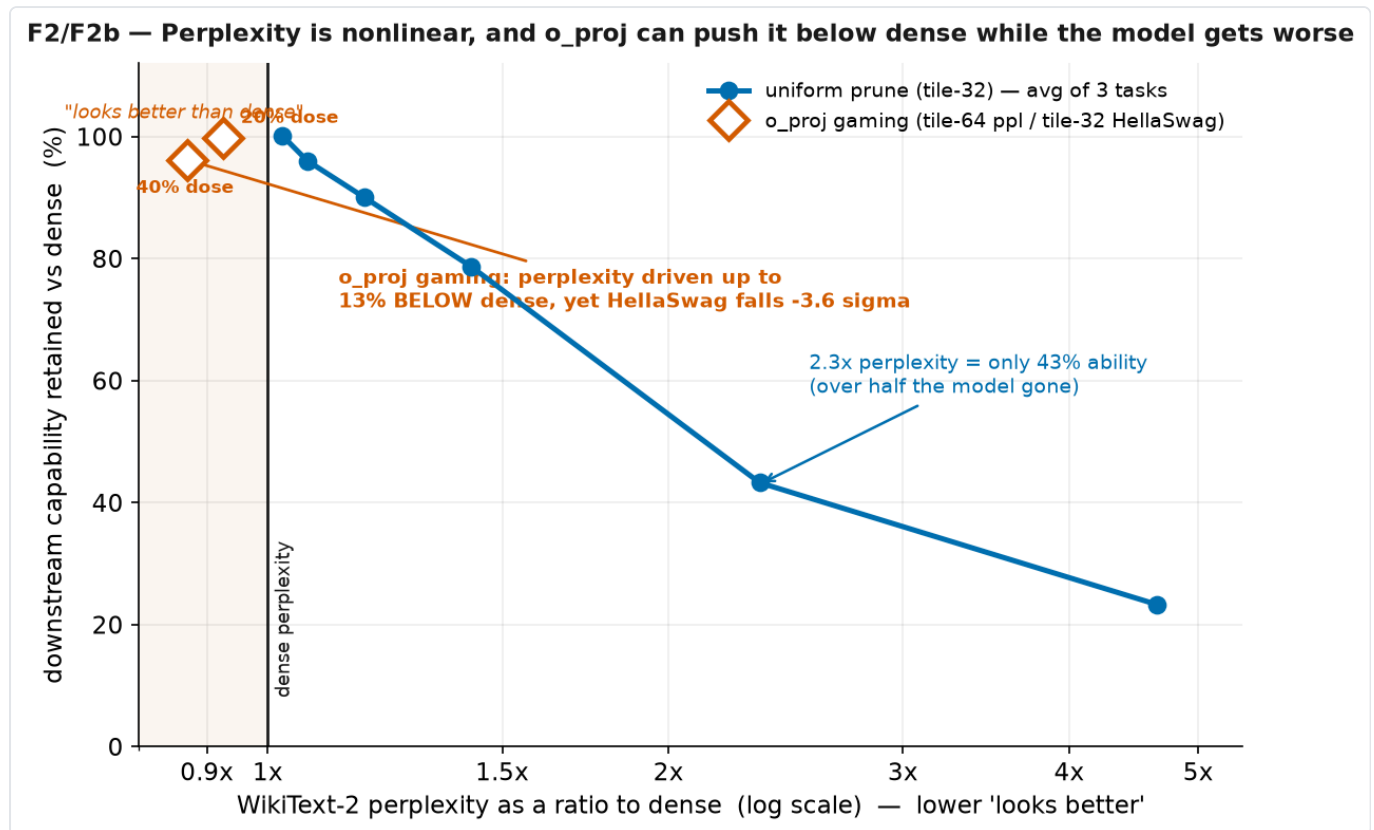
What we test. Whether you can deliberately make perplexity look *better* while real ability drops — by masking o_proj only in the layers where it helps in isolation.

Why we ran it. This is the sharpest possible statement of Finding 2, and the reason we ran ~11 downstream evals: if perplexity can be *driven the opposite direction* from capability, the field's default yardstick is not just loose but exploitable.

Result. Restricting Wanda o_proj masking to the 8 layers where o_proj improved in isolation (17–21, 32, 34, 35) drives WikiText perplexity **below dense** — 12.24 at 20% dose (–0.98) and **11.50 at 40% (–1.72, i.e. 13% "better" than dense)** — while real ability goes flat then down: at 40% HellaSwag drops a genuine **–3.6σ**, ARC-Easy –1.5σ, PIQA flat. Two controls make it airtight: blanket o_proj across all 36 layers never beats dense (14.61 at 20%), and SparseGPT **repair erases the perplexity win entirely** (13.29 / +0.07) — because repair reconstructs the dense output. The "gain" exists *only because you didn't reconstruct*: a textbook proxy exploit.

⚠ **Caveat (most attackable claim in the set):** two things to state plainly. (1) The perplexity-gaming numbers are archived **tile-64**; the downstream drop is **tile-32** ([run_downstream.py](#) re-prunes fresh at tile-32 and logs no perplexity). So the "two metrics move in opposite directions *in one network*" table is a **cross-tile composite** — each half is solid within its own tile size, but no *single* model was measured on both. Before presenting it as one network, run one tile-32 perplexity measurement on the exact gamed model (*scheduled* — see *Pending*). (2) The opposite-direction effect is carried by **one task at one dose** — HellaSwag at 40% (–3.6σ, ~1.6 pp absolute); PIQA is flat and ARC –1.5σ — and the below-dense perplexity magnitude is partly

select-on-WikiText / measure-on-WikiText overfitting. The *direction* and the controls are sound; the "one network" framing is the part not yet literally demonstrated.



capability-vs-perplexity-ratio scatter; the o_proj points sit left of the dense line ("looks improved") yet at/below dense ability.

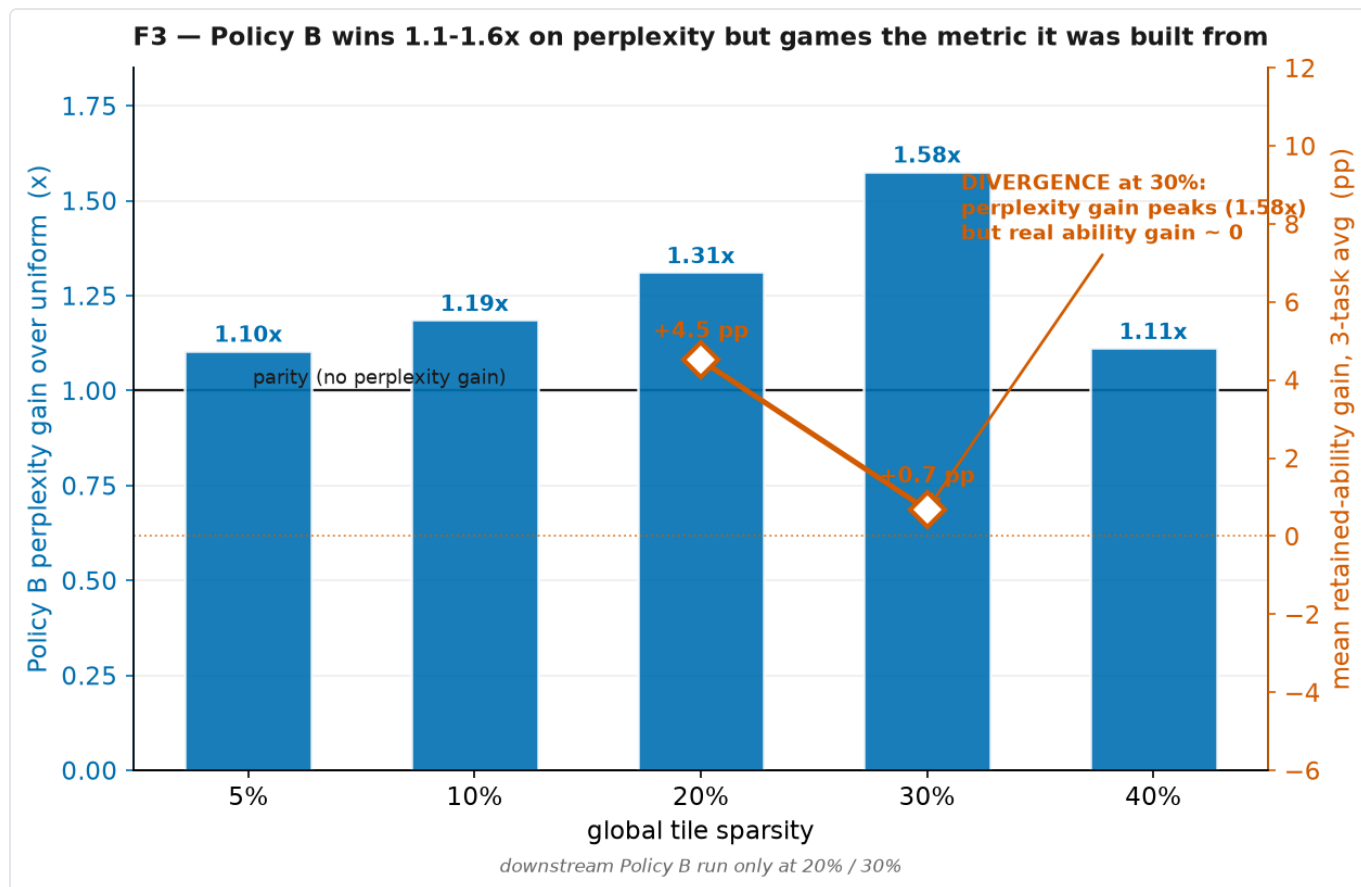
3 — "Smart" layer budgeting (Policy B) games the metric it was built from ★★ (downstream-anchored)

What we test. Whether a sensitivity-aware budget that protects fragile regions (Policy B) beats plain uniform pruning (Policy A) at the same compression — *and* whether its perplexity win shows up on real tasks.

Why we ran it. The sensitivity map was built *from perplexity*, so Policy B risks just flattering the number it was tuned on. Downstream accuracy at matched budgets separates a genuine win from a mirage.

Result. Both halves hold. Policy B genuinely edges uniform on perplexity by ~ 1.1 – $1.6\times$ (peaking $1.58\times$ at 30%), but that **badly overstates the capability win**: at its $1.58\times$ peak it buys essentially **zero** real ability ($+3.1/-0.6/-0.4$ pp = noise). It only wins for real at lower sparsity (20%: $+5.2/+1.3/+7.1$ pp). At *matched perplexity* it delivers ~ 10 – 18 pp **less** capability than uniform on all 6 measurements. Confirmed in metadata: it protects the perplexity-flagged "sensitive" class (pruned 0.042) and hammers the "robust" class (0.224) — and because that map came from perplexity (dominated by final layers feeding the LM head), it flatters the metric it was fit to.

Numbers (tile-32, sparsegpt_recon): ppl gain $1.10/1.19/1.31/1.58/1.11\times$ at 5/10/20/30/40%; downstream $\Delta(B-A) = +5.2/+1.3/+7.1$ pp at 20%, $+3.1/-0.6/-0.4$ pp at 30%.



perplexity-gain bars vs capability-gain line; they diverge at 30%.

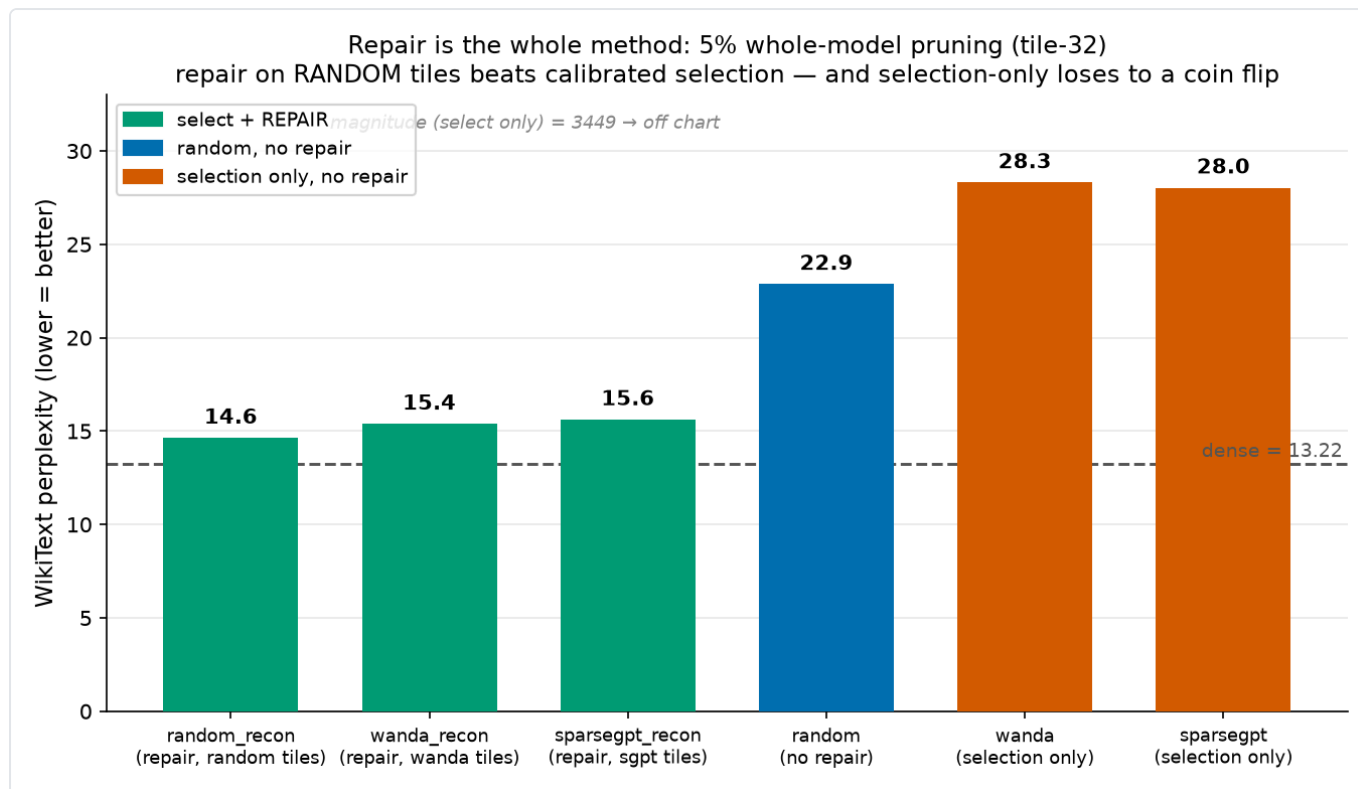
4 — Repair is everything; *which tiles you pick* barely matters ★ (control-relative)

What we test. Whether the model's recovery comes from *selecting* good tiles to delete or from *repairing* (least-squares reconstruction of) the surviving weights after deletion.

Why we ran it. These are the two levers. Knowing which one carries the method tells us where to spend effort — and it's the opposite of the field's instinct (which obsesses over selection).

Result (now proven with the full tile-32 paired ablation). Repair carries the entire method — and *which tiles you pick contributes nothing*. At 5% whole-model, **repairing after deleting random tiles (`random_recon` , 14.64) is the best result of all** — better than repairing after "smart" Wanda selection (`wanda_recon` 15.38) or SparseGPT selection (`sparsegpt_recon` 15.62), and all three land near dense (13.22). Selection *without* repair (Wanda 28.31, SparseGPT 28.03) is beaten by a plain coin flip (random 22.87). The causal proof is the same-tile pair: `random` and `random_recon` prune the **identical** tiles at the same seed, so adding repair alone drops perplexity **22.87 → 14.64**. Repair is the whole lever; calibrated selection not only fails to beat random, random-select-then-repair actually *edges out* calibrated-select-then-repair.

Numbers (tile-32, 5% whole-model, dense 13.22): repair — `random_recon` 14.64 (median of 5 seeds, 14.2–14.9), `wanda_recon` 15.38, `sparsegpt_recon` 15.62; no-repair — random 22.87; selection-only — wanda 28.31, sparsegpt 28.03; magnitude 3449.



repair methods (near dense) vs selection-only (2× worse) vs no-repair random, at 5%.

4b — Repair backfires exactly where it's needed most (final-layer MLP) (control-relative)

Result. At the deepest layer (L35), adding *more* calibration makes things *worse*, and the globally-best method — `sparsegpt_recon` — is the **single worst** choice there (dPPL 5.39 at 40%, vs random's 2.38). Repairing against a stale/ill-conditioned final-layer signal actively harms. This is the one place the "always repair" rule inverts.

4c / 4d — At the usable operating point, tile-selection loses to a coin flip ★★ (control-relative, corrected tile-32)

What we test. Whether data-aware tile scores (Wanda, SparseGPT) beat deleting *random* tiles — at 5% whole-model, and layer by layer.

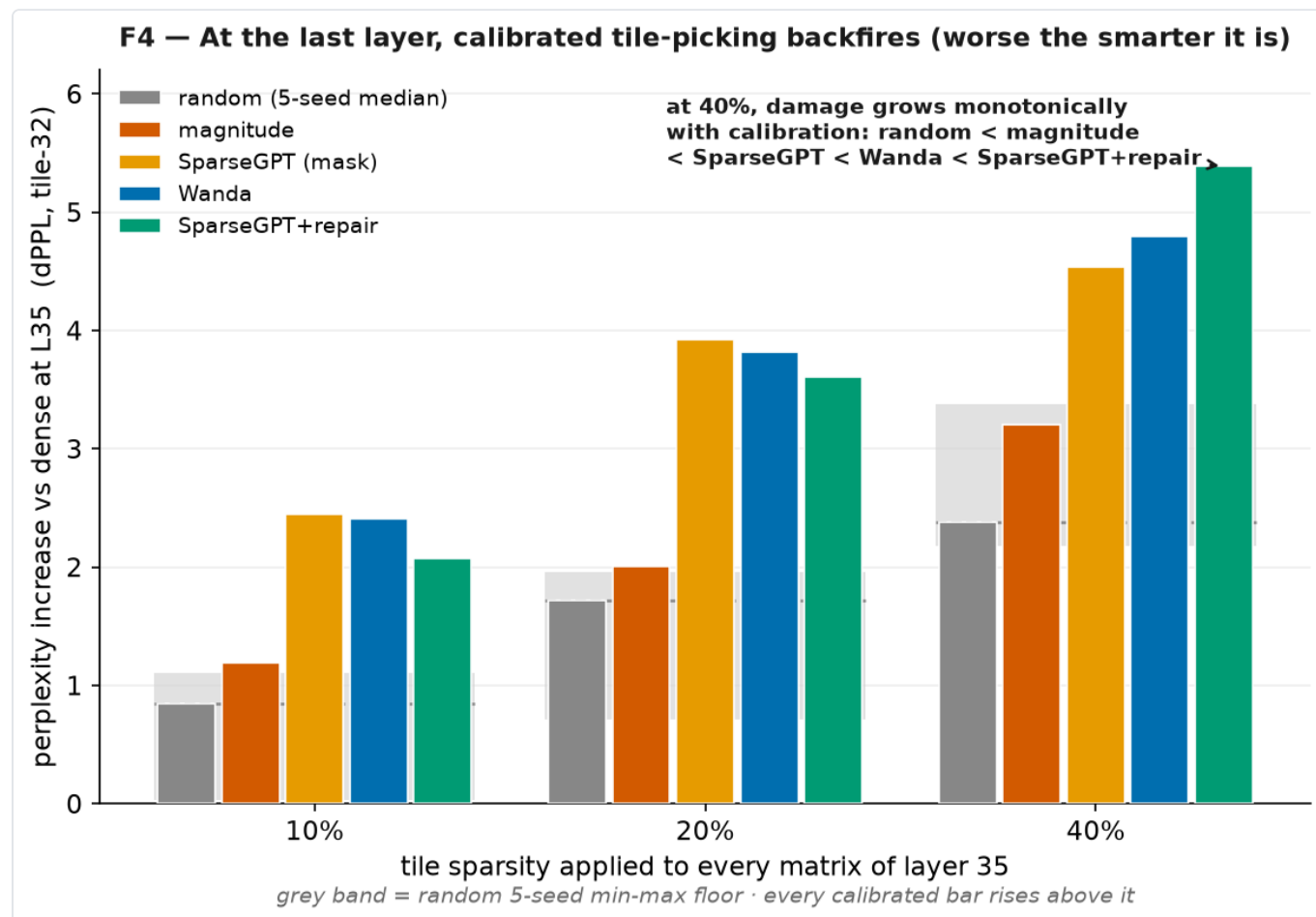
Why we ran it. A calibrated selection rule only earns its complexity if it beats chance. The earlier controls ran at the wrong tile size (64); we re-ran random + magnitude at tile-32 to give the claim a valid floor.

Result (sharper after correction). At **5% whole-model** — the setting where the pruned model still works — Wanda and SparseGPT each beat only **1 of 5** random seeds and lose to the random median (22.87). Per layer: **L0** data-aware is essential (random blows up at 40%); **middle layers** ~indistinguishable from random until 40%; **L35 selection backfires** — all 9 calibrated method×sparsity cells are worse than random ($z = +3.5$ to $+6.6$), and damage grows monotonically with how much calibration a method uses.

⚠ **Scope note (load-bearing — say it exactly this way):** "selection loses to random" is a claim about the **5% capability-preserving operating point** (the same 5% as Finding 1). Above it (10–20%), selection *does*

start beating random — but the model is already past the capability cliff there (random median 221 ppl at 10%, 178k at 20%), so it's a race between broken models. Consistent framing across Findings 1/4c/4d/8: *at the sparsity where capability is preserved (5%), calibrated selection is worthless; above it, everything without repair is degrading anyway*. Also note the honest **low power**: 5 seeds, and the calibrated points (wanda 28.3) sit inside the random spread (18–48), so this is "selection $\leq$ a coin flip," not a large-margin loss.

Numbers (tile-32): 5% whole-model — random seeds 18.0/20.2/22.9/26.7/48.0 (median 22.9); Wanda 28.3, SparseGPT 28.0 (each beat 1/5); magnitude 3,449 (0/5); sparsegpt_recon 15.6 (5/5). L35@40% dPPL: random 2.38 (best) < magnitude 3.20 < sparsegpt 4.53 < wanda 4.80 < sparsegpt_recon 5.39 (worst).



L35 dPPL by method ordered by calibration amount, random floor shaded.

5 — "Robust in isolation" does not compose: marginal safety $\neq$ joint safety

★ (downstream-anchored)

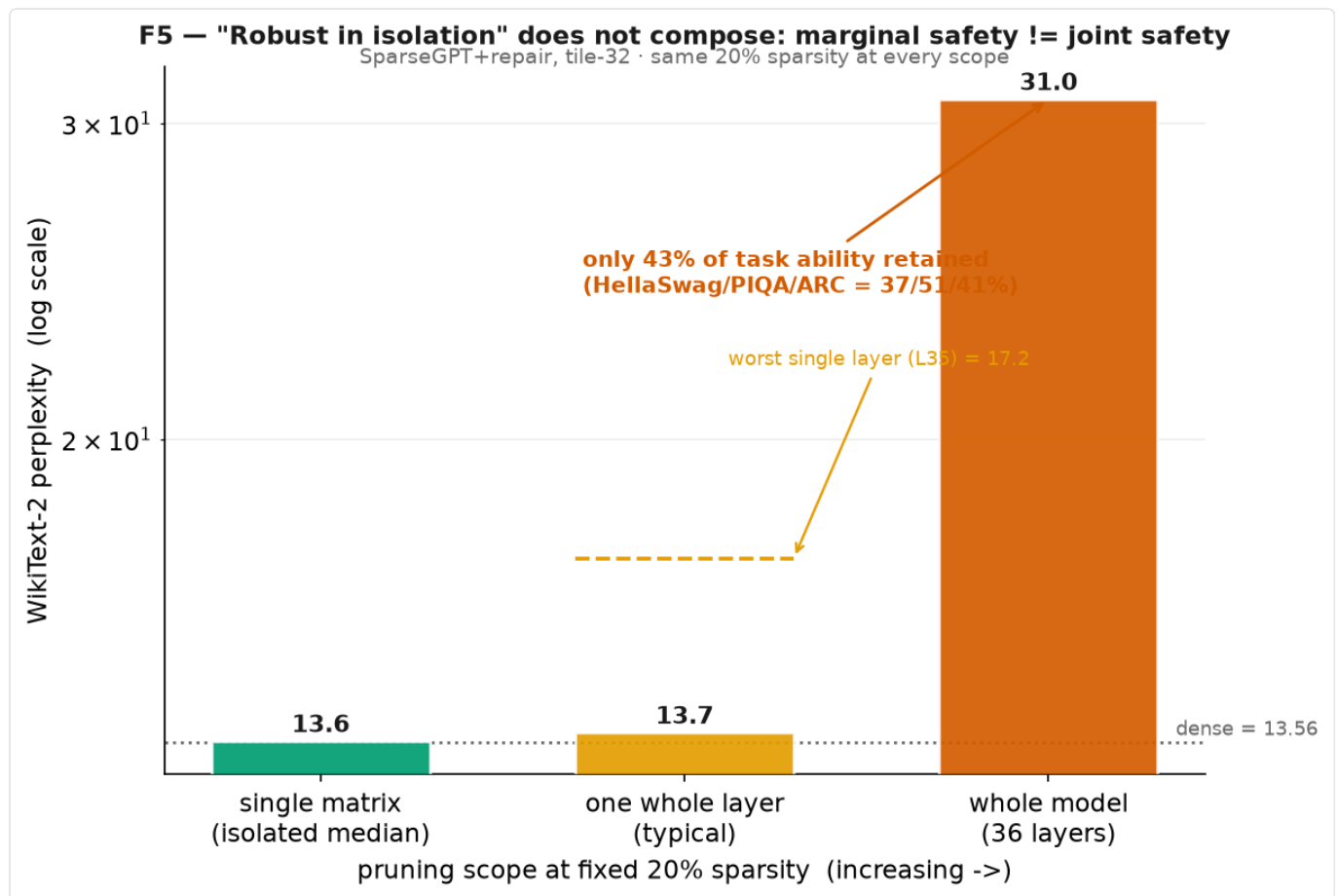
What we test. Whether a piece being prunable *on its own* tells you it's safe to prune once *many* pieces go at once.

Why we ran it. Our redundancy map is built one matrix at a time. The entire strategy leans on trusting that map jointly — this checks whether that assumption holds.

Result. It doesn't. One at a time, most of the model is harmless: at 20% sparsity 83% of screened matrices (with repair) barely move perplexity, and even a whole single layer's 7 matrices together stay near dense (except final-layer MLP). Yet pruning the **whole model** at 20% destroys most real ability (37/51/41% retained,

~60% gone). The map measures **marginal** damage; we were reading it as **joint**. The non-composition is an across-layer effect: 36 individually-fine hits compound. This is the quantified sequential-dependency limit.

Numbers (tile-32): isolated @20% — 83% of matrices $\Delta\text{PPL} < 0.12$; whole-layer joint @20% — L0/9/18/27 $\leq +0.4$ over dense, L35 = 17.16; whole-model joint @20% — ppl 30.95, ~43% ability retained.



perplexity vs pruning scope (matrix → layer → whole model), whole-model bar annotated "only 43% ability retained."

6 / 6b — Concentrating a fixed budget hurts monotonically; over-concentration collapses (control-relative, tile-32, ■ exploratory)

■ Exploratory follow-up — not in the adopted plan. We added it because the reallocation result (Finding 3) raised the question: given a fixed budget, is it better to spread cuts across many layers or pack them into fewer? Labeled exploratory to avoid dressing a post-hoc probe as pre-registered.

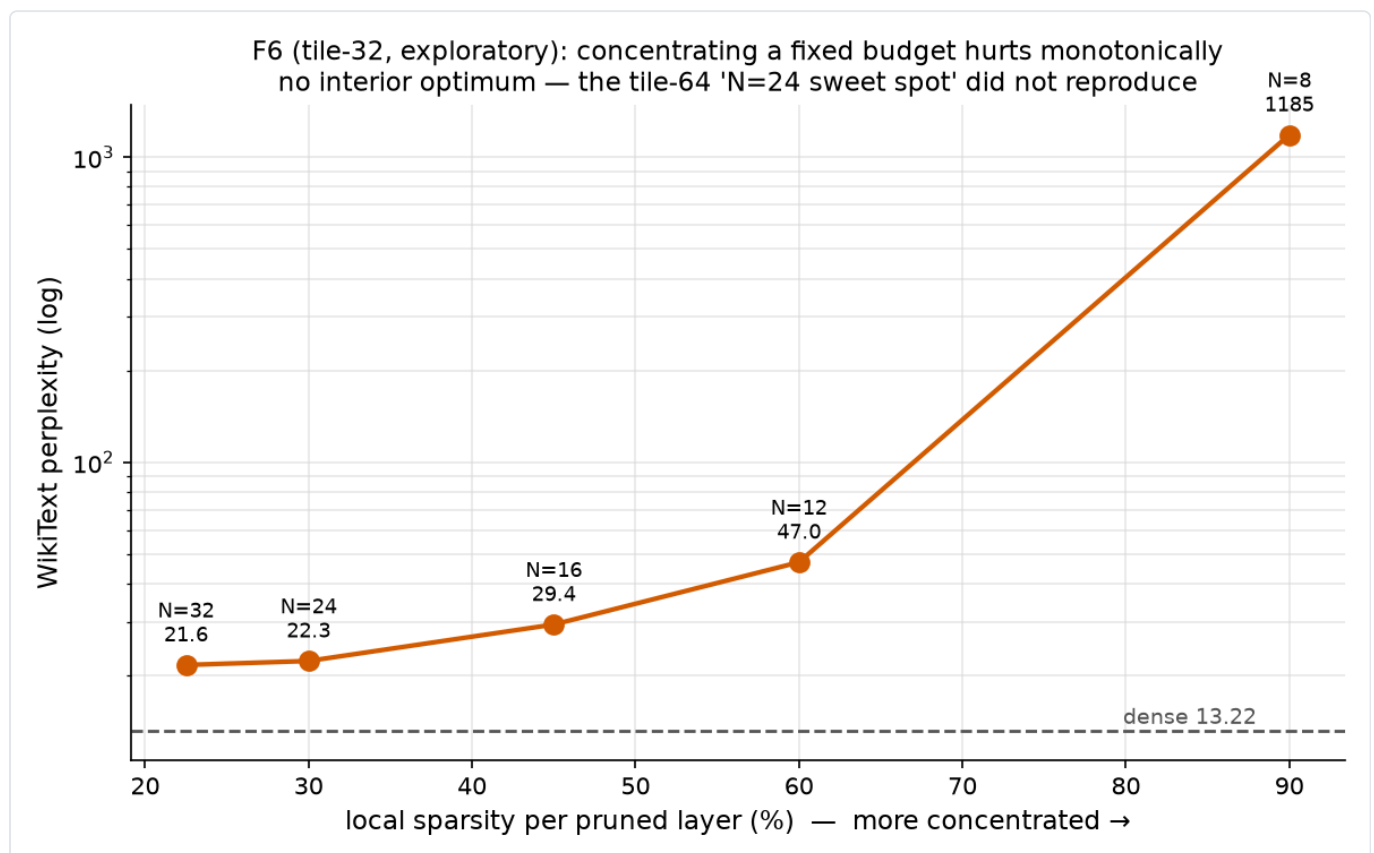
What we test. (6) Given a fixed tile budget, spread it thinly across many layers or pack it densely into fewer?
 (6b) When a layer gets one budget, does letting its 7 matrices share it unevenly beat uniform sparsity?

Result (re-run clean at tile-32 — corrects an earlier tile-64 artifact). The budget is packed into $N \in \{32, 24, 16, 12, 8\}$ evenly-spaced layers (the recovered original design; local sparsity rises as N falls to hold the budget fixed). At tile-32 the result is **monotonic** — the most-spread setting is best and quality degrades steadily as you concentrate: **21.65 (N=32) → 22.31 (N=24) → 29.38 (N=16) → 47.02 (N=12) → 1185 (N=8, destroyed).** There is no interior optimum. The earlier tile-64 run reported a "sweet spot" at $N=24$ (24.17, below $N=32$'s 24.94); that **reversed at tile-32** ($N=32$ now *beats* $N=24$) — it was a within-noise artifact, exactly

as suspected. The robust, keepable claim is only: *spreading a fixed budget is monotonically better than concentrating it, and extreme concentration collapses* — and that collapse is largely a restatement of Findings 5/9 (cross-layer compounding + the within-layer super-linear cliff), not independent evidence of a depth penalty.

For **6b** (layer-budget matching): a coin flip overall (7/15), and on inspection the allocation does **not** actually discover sensitivity (it prunes L18's *sensitive* matrices *more*, not less; Spearman $\rho \approx -0.15$). Its only real signal is the L35-vs-L0 effect-size contrast, not the win count. 6b is clean tile-32.

⚠ **Disclosed confounds (why this stays a secondary, exploratory finding):** (a) all depth configs prune only layers **0–31**, sparing the fragile tail 32–35 — so any "beats uniform" comparison is invalid (uniform prunes all 36 incl. the tail) and is **dropped**; (b) the over-concentration collapse is confounded with Finding 9's within-layer super-linear cliff (the fixed budget forces $N=8$ to 90% local sparsity). We present only the within-experiment monotonic trend, at tile-32.



perplexity vs concentration (log y) at tile-32: monotonic rise from the spread end ($N=32$) to the $N=8$ collapse, no interior minimum.

7 — Robustness is the depth × matrix-type *interaction*, not either alone

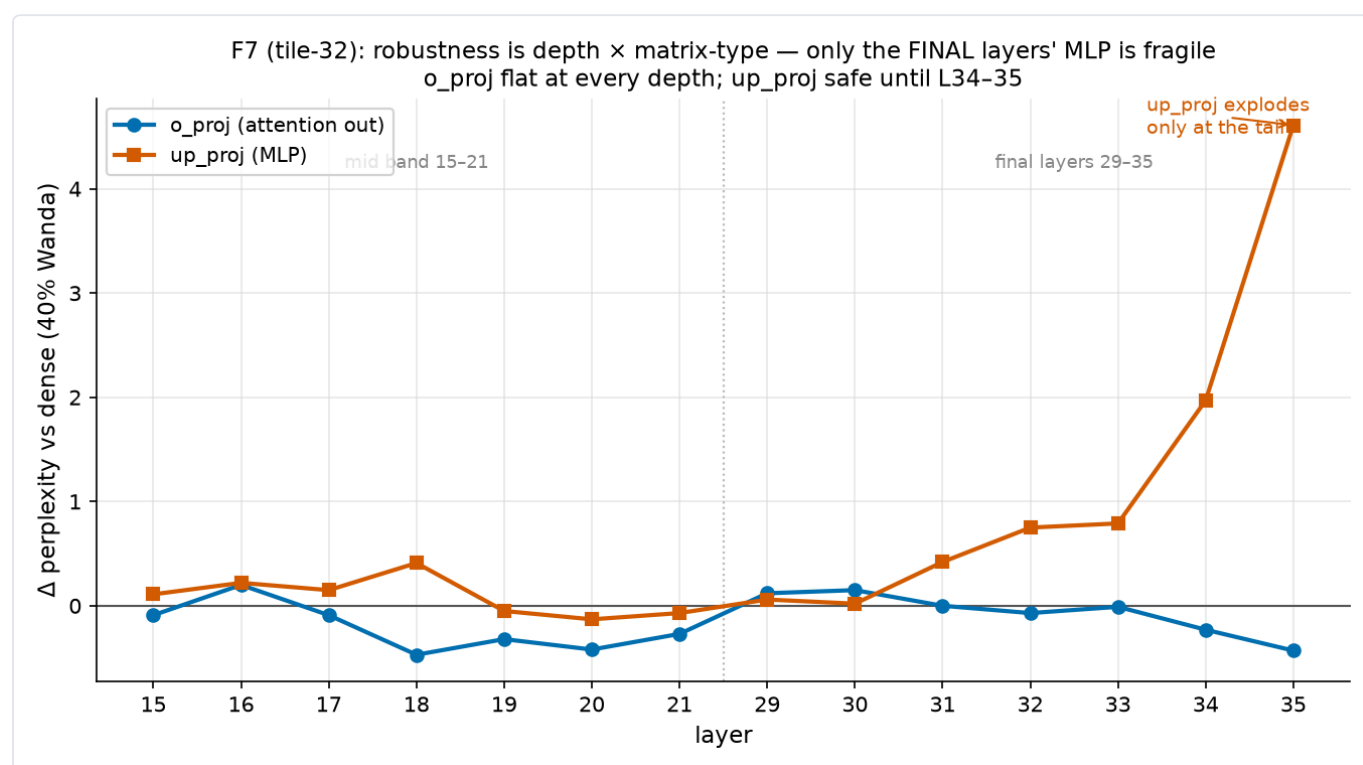
★ ★ (control-relative, tile-32, perplexity-only)

What we test. Whether prunability is set by depth, by matrix type, or specifically by the *combination* — pruning o_proj (attention-out) vs up_proj (MLP) across a mid band (15–21) and final layers (29–35).

Why we ran it. If it were purely "late layers fragile" or "MLP fragile," a one-line rule would suffice. If it's the pairing, every 1-D rule is wrong in principle and per-(layer, matrix) budgeting is justified.

Result (re-run clean at tile-32). On the perplexity axis, o_proj is safe to prune at *every* depth (flat-to-negative, even at the final layer), while up_proj is safe everywhere *except* the last one or two layers, where it explodes: at 40% Wanda, **up_proj L35 +4.61 dPPL** (L34 +1.97, L33 +0.79) vs **o_proj L35 -0.43**. So "late layers sensitive" and "MLP sensitive" are each false alone — only *the final layers' MLP* is. The pattern is essentially unchanged from the earlier tile-64 run (up_proj L35 was +4.70), so it's a real model property, not a tile artifact, and it replicates across Wanda (mask) and sparsegpt_recon (repair).

⚠ **Caveat + reconciliation (present as a marginal map, not a safety claim):** this is **perplexity-only, no downstream anchor**, and a *marginal* (one-matrix-at-a-time) result. Two honesty points the reviewers stressed: (i) "o_proj is safe" is certified by a perplexity *decrease* — the exact signal Finding 2b proves is **gameable** in the o_proj direction, so it is an internal tension unless scoped as marginal-only; (ii) "the interaction wins in all 6 cells" is **pseudo-replication** — 6 correlated views (2 methods × 3 sparsities) of a single late-layer up_proj effect, not 6 independent confirmations. Present this as a **marginal perplexity-sensitivity map**, not a deployment/safety recommendation. (Also: 2b's "8 o_proj-improving layers" come from the 14-layer cluster scan; the "32/36 improving" from the full-model scan — different scans, name both.)



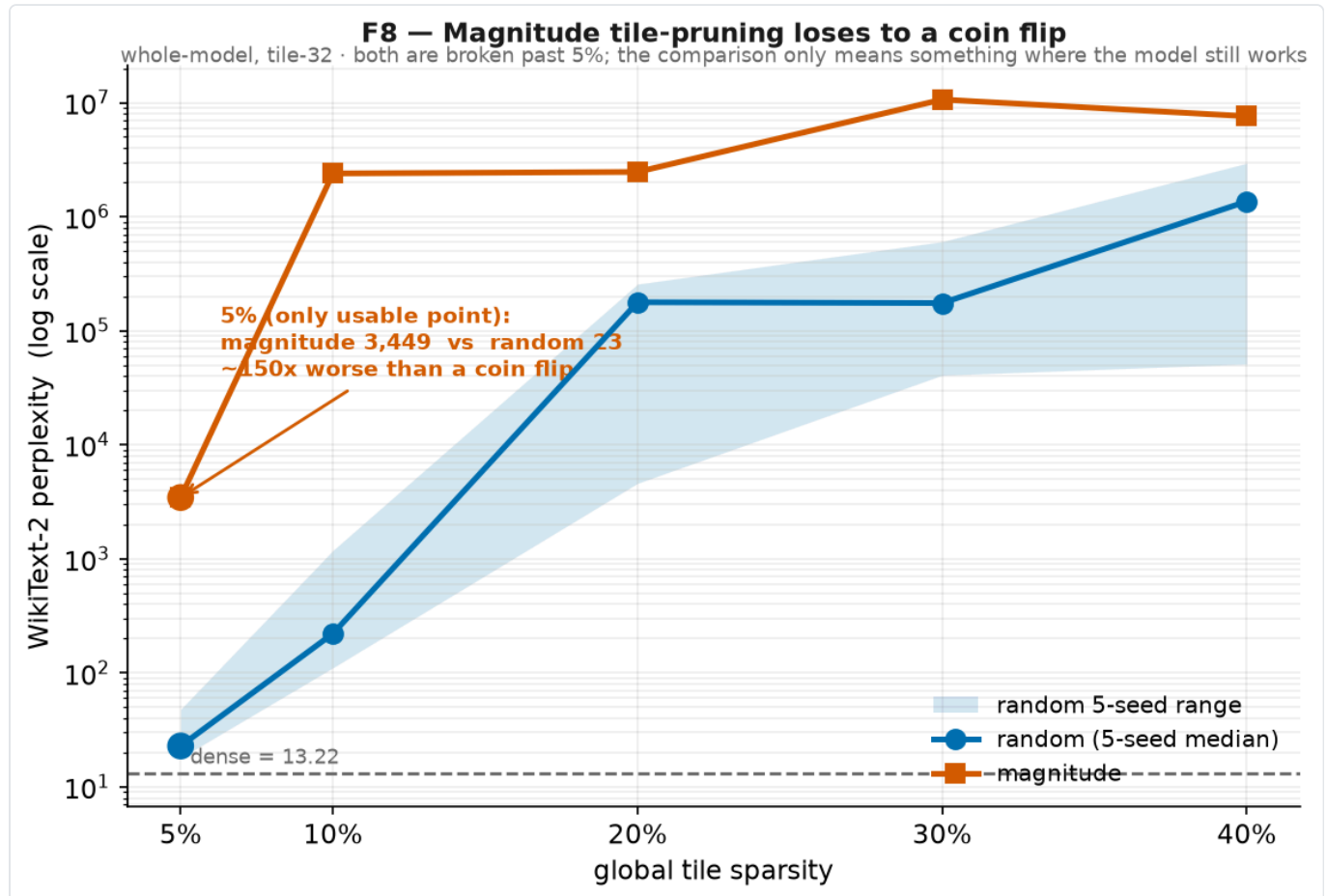
dPPL vs depth, o_proj vs up_proj lines fanning apart only at L34–35.

8 — Magnitude pruning is *worse than random* — and we know why (control-relative)

What we test. When deleting 32×32 tiles, is picking the smallest-magnitude tiles (the textbook default) better or worse than random?

Why we ran it. Magnitude is the obvious rule. If it can't even beat random block selection, tile pruning genuinely needs a smarter signal and naive intuition is a trap.

Result. At 5% (the capability-preserving point) magnitude **destroys the model** (ppl 3449 — well into the "destroyed" zone) while random stays roughly functional (median 22.9, $\sim 1.7\times$ dense). State this **qualitatively**: *at the sparsity where random still works, magnitude is already destroyed* — the exact " $\times N$ -worse" ratio is not meaningful because its numerator sits in the destroyed zone (>1000 ppl). The direction is robust and reproduces on the 0.6B model across 196 layer $\times$ matrix cells (random beats magnitude 144–52; the one consistent exception is o_proj). Mechanism (unifies with F9): per cell magnitude is only *slightly* worse than random, but those tiny deficits **compound across 36 layers** into the model-scale gap ("aggregation loss").



ppl (log) vs sparsity, magnitude vs random-median-with-band, 5% point annotated.

9 — Damage is sub-additive within a layer, compounding across layers (perplexity-absolute)

Result. Within a layer, damage overlaps (whole-layer ≈ 0.5 – $0.8\times$ the sum of its 7 matrices for the controls) → sub-additive. Across layers it super-compounds (whole-model damage runs $17\times$ to thousands $\times$ the sum of per-layer damage). **Caveats:** the exact 0.5 – 0.8 band is one slice — sub-additivity is method-dependent (data-aware methods sit near 1.0 , $\sim$ additive), L0 is a super-additive exception, and this is perplexity-absolute. F8 (a magnitude-vs-random comparison at identical settings) is robust to that caveat; F9 carries it.

Two things we are careful about (for the talk)

1. "Usable operating point" means the **capability-preserving point, 5%**. Findings 4c/4d/8 say selection/magnitude lose "at the usable point" — that point is 5% ($\sim 90\%$ ability retained). Finding 1's 10%

is *usable-but-degraded* (~79%), a different thing; above 5% the selection/magnitude comparisons flip or become races between broken models. We state the 5% scope on every affected finding.

2. **The o_proj / perplexity story is marginal, not a recommendation.** Finding 7's "o_proj is safe everywhere" is a *one-at-a-time, perplexity-only* observation (now tile-32); Finding 2b shows the joint, downstream-anchored reality (gaming) — and it uses the *same* perplexity signal 7 leans on. The two agree only once you separate marginal (7) from joint (2b) scope, so 7 is a sensitivity map, never a safety claim.

Pending experiments

★ The tile-size experiment — is 5% real, or an artifact of the 32×32 tile? (flagship)

The question. Every finding above is at 32×32, which is *large*. Unstructured pruning (1×1) recovers ~50% on LLMs; we get ~5%. So the ~5% ceiling — and "selection ≈ random" and "magnitude < random" — may be **artifacts of coarse granularity**, not real properties of Qwen3-4B. This experiment settles it. (*Designed by a 6-lens planning council.*)

The design. Treat tile size **T** as the **redundancy's correlation length** and measure it three ways that must agree: 1. **Purity probe** (Seb's front-end — *predicts*): a weight-space analysis that measures whether the removable 5% is spatially *clustered* (at some scale *S*) or *diffuse*. Mark the bottom-5% of weights by importance (Wanda + SparseGPT-saliency maps) and, for each *T*, measure the fraction of near-pure tiles vs a permutation null. Diffuse → smaller tiles always better, no sweet spot. Clustered at *S* → optimal tile ≈ *S*. Runs in minutes; sets the sweep priority. 2. **Perplexity/KL screen** (*previews*): fast proxy sweep, clearly labelled "not a capability claim." 3. **Downstream retained-ability ladder** (*decides*): the citable curve.

What's swept: new square sizes {16, 8, 4} + 1×1 (unstructured ceiling, separate vectorized code) — **32×32 is the existing reference, not re-run**. The model's matrix dims (2560/4096/1024/9728) divide every *T* exactly, so "% weights zeroed" = "% tiles zeroed" with **zero edge effects** (this is why 24 was ruled out — non-integer). Whole-model uniform, `sparsegpt_recon`, exact rungs {5,10,20}%, 5% first (the only point where capability is preserved).

Pre-registered outcomes: - **If coarse-granularity artifact:** as *T* shrinks, three findings move *together* — the ~5% ceiling rises, selection regains power over random, and magnitude<random vanishes — and all three crossovers coincide with the probe's correlation length *S*. → headline becomes "*redundancy is granularity-gated*," and we ship the structured-pruning tax curve. - **If real:** probe is diffuse and the downstream ladder stays flat across 32/16/8/4. → "*~5% is a real property of Qwen3-4B, not a tiling artifact*" — we ship that null.

Staged delivery (what you get first): the purity-probe verdict + a perplexity/KL screen at T16/T8 (T4 last) is a **complete first-pass answer**; the rigorous downstream ladder is launched and runs multi-day. One GPU job at a time, behind the recon refix.

Later (noted, not run): non-square tile shapes (1×*N* column strips) — the purity probe reports row-vs-column anisotropy, so it tells us *in advance* whether shape is worth chasing.

Other pending items

- **Close the 2b cross-tile gap:** one tile-32 perplexity run on the exact gamed o_proj model, so "opposite directions in one network" is literally one model (the most attackable claim → cheapest fix).
 - ~~Confirm Finding 6 at tile-32~~ ✓ **done** — re-run refuted the N=24 optimum (it's monotonic at tile-32); Finding 7 also re-run clean at tile-32.
 - **Recommended-method capability data:** wanda_recon / random_recon downstream ladder (finalising at tile-32 now).
 - **Second model (Llama-3.2-3B)** — the generality gate; every capability finding is n=1 model.
 - **Iterative / sequential calibration** — the one shot at genuinely raising the ~5% ceiling.
-

Where things live

- Active data (all tile-32): `experiments/{downstream, wholemodel, screen, screen_wholelayer, layer_budget, depth, screen_cluster}/` — `depth` (F6) and `screen_cluster` (F7) were re-run clean at tile-32.
- Archived tile-64 (superseded / closed threads): `experiments/archive/{full_scan, depth, oproj*, screen_cluster*}/` — the old tile-64 versions, kept for provenance.
- Figures: `figures/`
- Raw working log with full history: `FINDINGS.md`